AI Project Methodology – Graded Project: Customer Churn Prediction at RetailGenius

EPITA · MSc AI Systems

Marouane, Arman, Yahya, Prodipta

October 2026

Code repository: https://github.com/ArmanAhmed00/AI_Project_Methodology_ (shared with dinamedy@hotmail.com)

This report is split into three parts. In Part 1 we frame the churn project from the business and project management side. Part 2 covers how we actually built it: the code structure, the best practices we followed and how we used MLflow for tracking, packaging, the model registry and serving. In Part 3 we use SHAP to explain what our final model has learned.

Contents

- Part 1 – Functional Framing of the Churn Prediction Project
  - 1. Project Strategy
  - 2. Project Design
  - 3. Project Team
  - 4. Project Governance & Communication
  - 5. AI Project Management Methodology
- Part 2 – Technical Implementation
  - 1. Dataset
  - 2. Best practices we followed
  - 3. ML pipeline
  - 4. Experiments (MLflow tracking)
  - 5. MLflow Projects, Registry and Serving
- Part 3 – Explainable AI with SHAP
  - 1. Setup
  - 2. Explaining one customer (force plot)
  - 3. Explaining all customers at once
  - 4. Summary plot for each class
  - 5. Going further for one customer: waterfall plot
  - 6. Mean SHAP plot (global importance)
  - 7. Beeswarm plot
  - 8. Dependence plots (top 4 features)
  - 9. What this means for the business
- References

Part 1 – Functional Framing of the Churn Prediction Project

RetailGenius is a fast-growing international e-commerce marketplace. It wants to use its data to keep more of its customers. The first AI project in its data strategy is churn prediction, and this part is our framing of that project before any code is written.

1. Project Strategy

1.1 Strategic objectives

From the stakeholder interviews, we see four objectives for the project:

#	Objective	What it means in practice
O1	Reduce churn	Spot customers who are likely to leave early enough that we can still do something about it, and bring the churn rate down.
O2	Spend the retention budget better	Send vouchers, cashback and calls to the customers who are actually at risk and worth keeping, instead of to everyone.
O3	Understand why people leave	Give Product, Marketing and Customer Support the main reasons behind churn (complaints, short tenure, long delivery distance, and so on), not just a score.
O4	Build a base for future AI projects	Set up customer features, an MLOps pipeline and governance that later use cases (recommendations, fraud detection) can reuse.

The question we want the project to answer is: which active customers are likely to churn soon, why, and what action has the best chance of keeping them?

1.2 KPIs

We would follow the project at three levels. Model metrics tell us if the predictions are good. Business metrics tell us if the predictions actually save customers. Operational metrics tell us if the system works in production.

Level	KPI	Example target
Model	ROC-AUC and PR-AUC on the hold-out set	AUC of 0.90 or more
Model	Recall on churners (share of real churners we catch)	80 % or more
Model	Precision among the top-k customers we score (limited by campaign capacity)	60 % or more
Model	Calibration (Brier score) and score stability (PSI)	PSI below 0.2
Business	Churn rate of contacted customers compared with a control group (A/B test)	15 % lower
Business	Retention rate and repeat-purchase rate	+5 points
Business	Customer lifetime value saved minus campaign cost (ROI)	ROI above 3
Business	CSAT / NPS of contacted customers	should not drop

Level	KPI	Example target
Operational	API latency, freshness of batch scores	under 200 ms, updated daily
Operational	Adoption: share of retention campaigns that use the churn score	above 80 %

1.3 How AI can help retention

Today, RetailGenius mostly notices churn after the customer has already gone. With a model, we can score customers before that happens and act while there is still time.

The model also lets us personalise the response. If we combine the risk score with the main reasons behind it (we get these from SHAP, see Part 3), a customer who complained can get a call from support, while a price-sensitive customer gets a coupon. Ranking customers by risk times value means a limited budget goes where it brings the most back.

At a higher level, the explanations show structural problems, such as weak onboarding or slow complaint handling, that Product can fix for everyone. Finally, the results of each campaign come back as new labels, so the model keeps improving over time.

2. Project Design

2.1 Data

Relevant data sources

Domain	Source	Example features
Customer profile	CRM (SQL)	tenure, city tier, gender, marital status, number of addresses
Behaviour	Platform logs / clickstream (NoSQL, data lake)	hours on the app, devices, sessions, product views, searches
Transactions	Order system	number of orders, days since last order, order amount growth, preferred category, payment mode
Marketing	Campaign tool	coupons used, cashback, email opens, campaign responses
Customer support	Ticketing tool	complaints, number of tickets, resolution time, satisfaction score
Logistics	Warehouse and delivery systems	warehouse-to-home distance, late deliveries, returns
External	Review sites, social media, competitor prices, economic and demographic data	review sentiment, price gap with competitors, regional income

Before anything else, the business has to agree on what churn means. A simple definition would be “no purchase and no login for 90 days”. The label drives everything after it, so this needs to be decided and signed off early.

Challenges and how we would handle them

Challenge	Why it matters	How we would handle it
Data silos (CRM, logs, support, marketing)	We don't get a full view of the customer	A common customer ID, ETL with NiFi/Talend into the data lake, and a feature store
Data quality: missing values, inconsistent labels ("Phone" vs "Mobile Phone"), outliers	The model learns from bad data	Automatic data quality checks (for example Great Expectations), imputation, written cleaning rules
Unclear label and data leakage	Results look better than they really are	A clear churn definition, a time-based split, only features known before the prediction date
Class imbalance (about 17 % churners)	The model can ignore the minority class	Stratified splits, class weights, PR-AUC as a metric, threshold tuning
Privacy (GDPR)	Legal risk	Data minimisation, pseudonymisation, a clear legal basis, a DPIA, no sensitive attributes, access control
Fairness	Some groups could be treated unfairly	A bias check across gender and region; protected attributes are not used for targeting
Data volume	Pipelines become slow	Distributed processing (Spark) and incremental feature computation

2.2 Models

Which models make sense

Model	Strengths	Weaknesses	Role in the project
Logistic regression	Simple, fast, easy to explain	Only captures linear effects	Baseline
Random Forest	Robust, handles non-linear effects	Bigger, probabilities less well calibrated	Challenger
Gradient boosting (XGBoost, LightGBM, CatBoost)	Usually the best on tabular data, handles missing values, works well with SHAP	Needs some tuning	Main candidate
Survival models (Cox, survival forests)	Predict *when* a customer will leave	Labels are more complex	Later iteration
Neural networks on clickstream sequences	Can use raw behaviour	Need a lot of data, hard to explain	Future work

Training, validation and testing

We would start by splitting the data into a stratified training set and test set (80/20). In production we would also keep an out-of-time test set: train on older months and test on the most recent one, since that's closer to how the model will be used.

On the training set, we use stratified 5-fold cross-validation to compare models and tune hyper-parameters (grid search, random search or Optuna). The test set is used only once at the end. There we look at ROC-AUC, PR-AUC, recall, precision, F1 and calibration, and at a business metric such as the expected profit at the chosen threshold.

The threshold itself is decided with Marketing, because it depends on how many customers they can contact and on what a false alarm costs compared with a missed churner. Before going live, we also check the explanations (SHAP) and fairness.

Versioning and serving

- Code: Git and GitHub, with feature branches, pull requests, code review and CI that runs the linter and tests.
- Data: dated snapshots in the data lake (or DVC). We record the dataset version with each run.
- Experiments and models: MLflow Tracking for parameters, metrics and artifacts, and the MLflow Model Registry for versions, aliases (`@champion`, `@challenger`) and approval tags.
- Serving: the model is packaged as an MLflow model and used in two ways. A daily batch job writes the scores into the CRM, and a REST API (MLflow serving or FastAPI in Docker) handles real-time calls, for example from the support tool.

2.3 Deployment

Strategy	How it works	When we would use it
Batch scoring	Score all customers every day or week and push the results to the CRM	Default option for retention campaigns
Real-time API	Score a customer on request	Support tickets, in-app offers
Shadow deployment	The new model scores in parallel but nobody acts on its output	Before replacing a model
Canary / A-B test	The new model is used for a small share of customers first	To measure the real business uplift
Blue-green	Two environments with an instant switch and rollback	Releases without downtime

For production, there are a few more things to think about:

- Scalability: containers, Kubernetes, autoscaling.
- Security: authentication, encryption, secrets management.
- GDPR: the customer's right to object to profiling, and data retention rules.
- Monitoring and logging, plus a clear rollback plan.
- A CI/CD pipeline: tests, image build, staging, approval, then production.
- Running costs, and integration with the existing SQL/NoSQL systems and the CRM.
- A model card that documents what the model does and where its limits are.

2.4 Monitoring

What we watch	Metric	How often	When we raise an alert
Data quality	schema, % missing, value ranges	every batch	the pipeline fails
Data drift	PSI or KS test per feature	weekly	PSI above 0.2
Prediction drift	distribution of scores, % of customers flagged	weekly	±30 % compared with the reference
Model performance	AUC, recall, precision, once labels are known (about 90 days later)	monthly	AUC drops by more than 5 points

What we watch	Metric	How often	When we raise an alert
Business impact	churn of the treated group vs. the control group, ROI	monthly	uplift no longer significant
Service	latency, error rate, uptime	real time	SLO breached

What we do when the model drifts

1. Detect it with automatic drift reports (for example Evidently AI) and a dashboard.
2. Find the cause. Is it a data problem, such as a broken pipeline, or has customer behaviour really changed (a new competitor, a seasonal effect)?
3. Fix the data problem, or retrain on recent data. We plan a monthly retrain anyway, plus an extra one when an alert fires.
4. Validate the new model. The challenger has to beat the current champion on a recent out-of-time set and pass the fairness checks.
5. Release it through shadow mode or a canary first, then move the `@champion` alias in the registry. The previous version stays available for rollback.

3. Project Team

3.1 Roles and skills

Role	What they do	Key skills
Business sponsor (e.g. Head of CRM)	Sets the objectives, owns the budget and the KPIs	Retention strategy, decision-making
Product owner	Manages the backlog and priorities, links business and tech	Agile, business analysis, communication
Project manager / Scrum master	Planning, risks, ceremonies, reporting	Scrum, risk management
Data engineer	Ingestion, ETL, data quality, feature store	SQL, Spark, NiFi/Talend, cloud, data modelling
Data scientist	EDA, features, modelling, evaluation, explainability	Python, ML, statistics, SHAP
ML engineer / MLOps	Packaging, CI/CD, serving, monitoring	Docker, MLflow, Kubernetes, APIs
Data analyst	KPIs, dashboards, campaign analysis	BI tools, SQL, A/B testing
Integration engineer	Connects the model to the CRM, the app and the support tool	APIs, back-end development
Data governance / DPO	GDPR compliance, data access, ethics	Privacy law, governance
Business experts (Marketing, Customer Support)	Bring domain knowledge and feature ideas, act on the scores	Knowing the customers

3.2 Collaboration and alignment

We would run this as one cross-functional squad rather than separate business and tech teams. Everyone takes part in the same Scrum ceremonies and works from the same Trello/Jira board. Shared tools help a lot here: GitHub, MLflow, a shared drive or Confluence, a data dictionary, and above all one written definition of churn that everybody uses.

To stay aligned with the strategy, the team's OKRs come straight from objectives O1 to O4, and each sprint goal is linked to a KPI. Stakeholders see a demo at every sprint review. A task only counts as done once the business has validated it.

Marketing and Customer Support are the ones who will actually use the predictions, so they need to be involved from the start. We would design the retention actions with them (one playbook per main churn reason), run the A/B tests together, and hold a short weekly sync. Campaign results and feedback from support agents come back to the data team. The scores go into the tools they already use, such as a field in the CRM or the support console, rather than a new app they would have to open.

4. Project Governance & Communication

4.1 Stakeholders and communication plan

Stakeholder	Interest / influence	What they need from us	Channel	Frequency
Executive committee / sponsor	High / High	ROI, progress, risks, go/no-go decisions	Steering committee, one-page dashboard	Monthly
Marketing executives	High / High	Lists of at-risk customers, churn reasons, campaign results	Sprint review, BI dashboard	Every two weeks
Product managers	High / Medium	Churn reasons linked to features and UX	Workshops, reports	Monthly
Customer support	Medium / Medium	Risk level and reasons for each customer, inside their tool	Training, CRM field, FAQ	At launch, then monthly
Data team	High / High	Requirements, data access	Daily stand-up, Jira/Trello	Daily
Tech / infrastructure	Medium / High	Integration, security, scalability	Architecture review	At each milestone
DPO / legal	Medium / High	GDPR compliance	DPIA review	At each milestone
Finance	Medium / Medium	Costs and ROI	Steering committee	Quarterly

4.2 Governance bodies

Body	Who is in it	Purpose	Frequency
Steering committee	Sponsor, heads of Marketing, Tech and Data, product owner	Strategy, budget, go/no-go, major risks	Monthly
Project committee	Product owner, project manager, leads	Planning, risks, dependencies	Weekly
Sprint ceremonies	The squad and business experts	Planning, daily stand-up, review/demo, retrospective	Two-week sprints
Architecture board	Tech lead, data engineer, security	Technical choices and integration	At each milestone

Body	Who is in it	Purpose	Frequency
Model risk and ethics review	Data science lead, DPO, business	Validate the model, fairness and explanations before go-live	Before each release
Data governance council	Data owners, DPO	Data quality, access, lineage	Monthly

4.3 Communicating model outputs

Technical and non-technical people need very different things from the model.

For the technical teams, we share the MLflow UI (runs, metrics, model versions), the Sphinx documentation, a model card (data used, metrics, known limits), the ROC/PR curves and confusion matrix, the SHAP plots and the drift reports.

For everyone else, we keep it simple. Instead of a raw probability, each customer gets a risk level: High, Medium or Low. Next to it, we show the three main reasons in plain language, taken from SHAP: for example “complained recently, new customer, many delivery addresses”. We add a suggested action and a dashboard with the business KPIs (customers saved, ROI). Short training sessions and an FAQ explain what the score means, and also what it does not mean.

5. AI Project Management Methodology

5.1 Our choice: CRISP-DM inside an Agile (Scrum + Kanban) setup

CRISP-DM gives us the steps of a data science project: business understanding, data understanding, data preparation, modelling, evaluation and deployment, with the possibility to loop back. Scrum gives the rhythm, with two-week sprints, regular demos and feedback from stakeholders. A Kanban board (Backlog, To do, In progress, Review, Done) makes the flow of tasks visible and limits how much is in progress at once. After deployment, MLOps practices (monitoring, retraining) keep the model alive.

5.2 Why this fits the project

AI projects are uncertain by nature. At the start we don’t know which data or which model will work, so a fixed waterfall plan would not hold for long. Short iterations work better.

CRISP-DM accepts that we will go back from evaluation to business or data understanding, which is what really happens in ML projects. Frequent demos keep Marketing and Support involved and help us catch misunderstandings early, like a disagreement on the churn definition. This setup also lets us deliver a first useful version quickly (a baseline model with batch scoring) and improve it step by step.

5.3 Risks and mitigation

Risk	Probability	Impact	Mitigation
Poor data quality or data silos	High	High	Data audit in sprint 0, data quality rules, a data engineer in the squad
Wrong or unclear churn definition	Medium	High	Workshop with the business at the start and formal sign-off
Data leakage making results look too good	Medium	High	Time-based split and a review of every feature

Risk	Probability	Impact	Mitigation
The business doesn't use the model	Medium	High	Design it with them, explain the predictions, integrate into existing tools, train users
GDPR or ethics problem	Low	High	DPO involved early, DPIA, fairness audit
Model drift after deployment	High	Medium	Monitoring and a retraining plan
Scope creep (recommendations, fraud, ...)	Medium	Medium	A clear MVP scope and a backlog prioritised by the product owner
Too much depends on one person	Medium	Medium	Documentation, pair work, code reviews
Cost overrun (cloud, extra iterations)	Medium	Medium	Timeboxed experiments and budget tracking

5.4 Handling costs and planning drift caused by model iterations

Model work can go on forever if nobody sets limits, so we would put some guardrails in place:

- Timebox each experiment, for example one sprint per modelling idea, with a clear go/no-go criterion at the end.
- Agree with the business on what "good enough" means (for example AUC of 0.90 and recall of 80 %), and stop optimising once we get there.
- Ship a simple baseline early, so the business gets value while we keep improving the model.
- Keep a buffer of about 15–20 % in the budget and planning for the uncertainty that comes with AI work.
- Track the compute cost of each experiment, and choose the cheaper model when the gain is small.
- Re-plan at every sprint review based on velocity, and take bigger deviations to the steering committee.

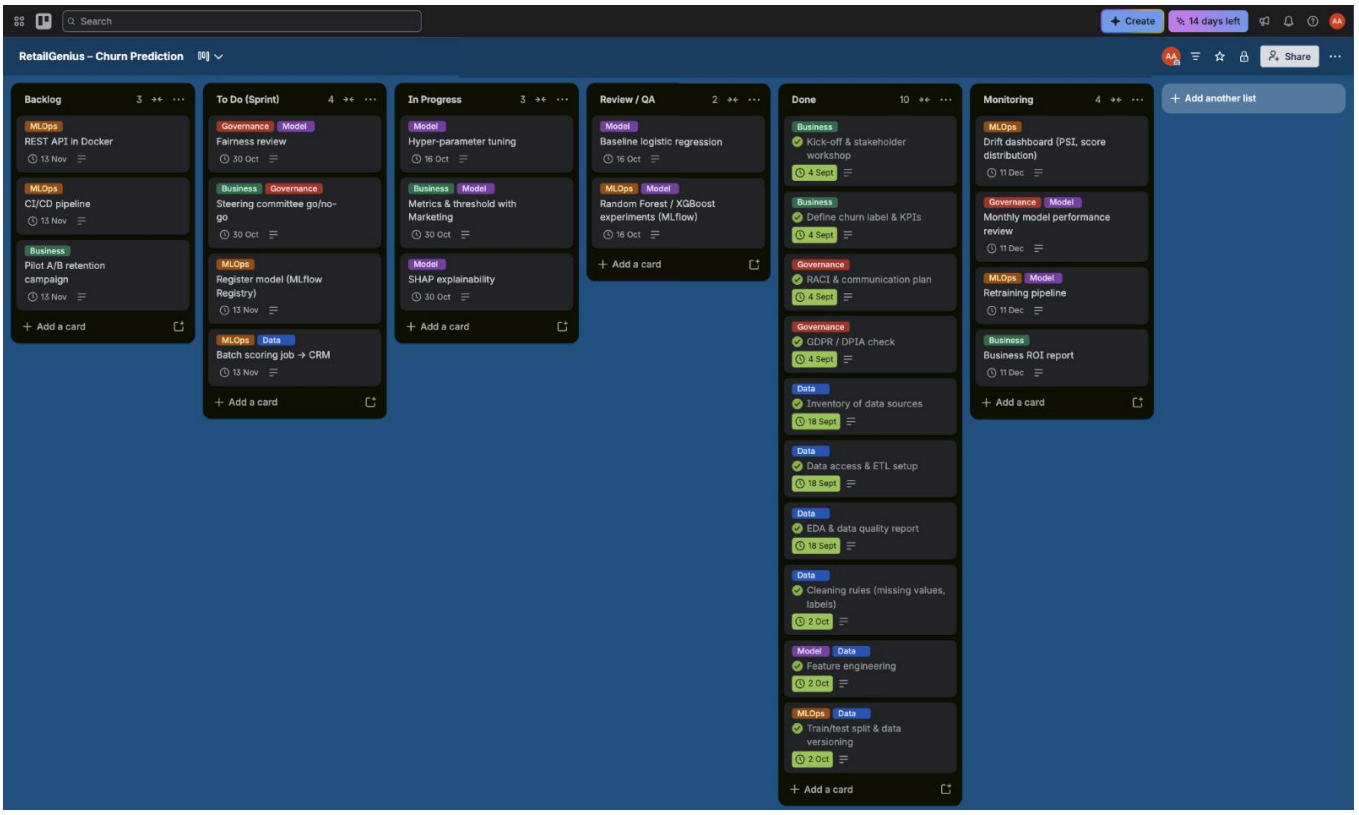
5.5 Mock project on Trello (Kanban)

We set up a board called "RetailGenius – Churn Prediction" with six lists: Backlog, To Do (Sprint), In Progress, Review / QA, Done and Monitoring. Labels show the type of work (Business, Data, Model, MLOps, Governance), and each card has an owner, a due date and a checklist.

Phase (CRISP-DM)	Cards
Sprint 0 – Business understanding	Kick-off and stakeholder workshop · Define the churn label and KPIs · RACI and communication plan · GDPR / DPIA check
Sprint 1 – Data understanding	Inventory of data sources · Data access and ETL setup · EDA and data quality report
Sprint 2 – Data preparation	Cleaning rules (missing values, labels) · Feature engineering · Train/test split and data versioning
Sprint 3 – Modelling	Baseline logistic regression · Random Forest / XGBoost experiments in MLflow · Hyper-parameter tuning
Sprint 4 – Evaluation	Metrics and threshold agreed with Marketing · SHAP explanations · Fairness review · Steering committee go/no-go
Sprint 5 – Deployment	Register the model in the MLflow Registry · Batch scoring job to the CRM · REST API in Docker · CI/CD pipeline · Pilot A/B campaign

Phase (CRISP-DM)	Cards
Run – Monitoring	Drift dashboard · Monthly performance review · Retraining pipeline · Business ROI report

The screenshot below shows the board in the middle of the project: sprints 0 to 2 are done, modelling and evaluation are in progress or in review, and deployment and monitoring are still planned. The board is available at https://trello.com/b/x7zR3Fet/retailgenius-churn-prediction.



Our Trello board for the churn prediction project

Part 2 – Technical Implementation

Code repository: https://github.com/ArmanAhmed00/AI_Project_Methodology_ (shared with dinamedy@hotmail.com)

In this part we turn the framing from Part 1 into a working project. As the brief says, the goal was not the best possible score. We wanted a project that is clean, reproducible and close to what a team would put into production.

1. Dataset

We used the E-Commerce Customer Churn Analysis and Prediction dataset from Kaggle (sheet `E Comm`). It has 5,630 customers and 20 columns, with a binary target `Churn`. About 16.8 % of customers churned, so the classes are imbalanced.

Our EDA (`notebooks/01_eda.ipynb`) found three issues to deal with:

- Seven numerical columns have missing values, each around 4.5–5.5 %: `DaySinceLastOrder` (307), `OrderAmountHikeFromlastYear` (265), `Tenure` (264), `OrderCount` (258), `CouponUsed` (256), `HourSpendOnApp` (255) and `WarehouseToHome` (251). We impute them inside the model pipeline (median for numbers, most frequent value for categories), so exactly the same logic runs at serving time.
- Some categories appear under two names. We merged `Phone` into `Mobile Phone`, `CC` into `Credit Card`, `COD` into `Cash on Delivery` and `Mobile` into `Mobile Phone`.
- `CustomerID` is just an identifier, so we never use it as a feature.

2. Best practices we followed

Requirement	What we did
Versioning and collaboration	Git and GitHub, with feature branches merged into `main`. Models are versioned in the MLflow registry and exported to `models/`.
Code structure	A layout adapted from cookiecutter-data-science: `data/raw`, `data/interim`, `data/processed`, `models`, `notebooks`, `reports`, `docs`, `src`, `tests`
Separate workflow scripts	`data.py` (preparation), `features.py` (feature engineering), `train.py` (training), `register.py` (registry), `predict.py` (inference), `explain.py` (XAI)
Standardisation	Everyone in the group uses Python 3.13 (`.python-version`). Code follows PEP8, checked with black (formatter) and flake8 (linter) through `make lint`.
Reproducibility	Poetry with pinned versions in `pyproject.toml` and `poetry.lock`, a `requirements.txt` for pip users, fixed random seeds, and `python_env.yaml` for MLflow Projects
Documentation	Sphinx (autodoc, napoleon, autosummary, Read-the-Docs theme) builds the docs from our Google-style docstrings with `make docs`
Tests	pytest unit tests for cleaning, feature engineering and the full pipeline
Tracking	MLflow Tracking logs parameters, CV and test metrics, ROC and confusion-matrix figures, and the model with its signature and an input example
Packaging	MLflow Projects (`MLproject`) with the entry points `prepare`, `train`, `sweep`, `register`, `explain` and `main`

Requirement	What we did
Model registry	MLflow Model Registry: model `churn-classifier`, several versions, the `@champion` alias, a description and a `validation_status` tag
Serving	MLflow serving: a local REST API with `mlflow models serve` on port 5001
Bonus (cloud)	A `Dockerfile` that serves the exported champion model. It builds locally and is ready for Cloud Run, Render or Azure Container Apps.

3. ML pipeline

```
raw xlsx  ->  data.py: remove duplicates, fix labels, stratified 80/20 split
          ->  Pipeline[ FeatureBuilder -> ColumnTransformer (impute + one-hot) -> classifier ]
          ->  5-fold stratified CV + hold-out test  ->  logged to MLflow
```

We added three engineered features in `FeatureBuilder`:

- `CashbackPerOrder`: cashback divided by the number of orders.
- `CouponPerOrder`: coupons divided by the number of orders.
- `IsNewCustomer`: 1 if tenure is one month or less.

All the preprocessing lives in a single scikit-learn `Pipeline`. That means the logged model takes raw customer records as input, and we avoid differences between training and serving.

We save the model in MLflow's skops format, which is safer than pickle, with an explicit list of trusted types. Numerical inputs are cast to float so that the model signature still accepts missing values at prediction time.

4. Experiments (MLflow tracking)

`make sweep` runs six experiments: a logistic regression baseline, two Random Forests and three XGBoost configurations. Each run logs its hyper-parameters, the mean and standard deviation of the CV ROC-AUC, and the ROC-AUC, PR-AUC, F1, precision, recall and accuracy on the train and test sets.

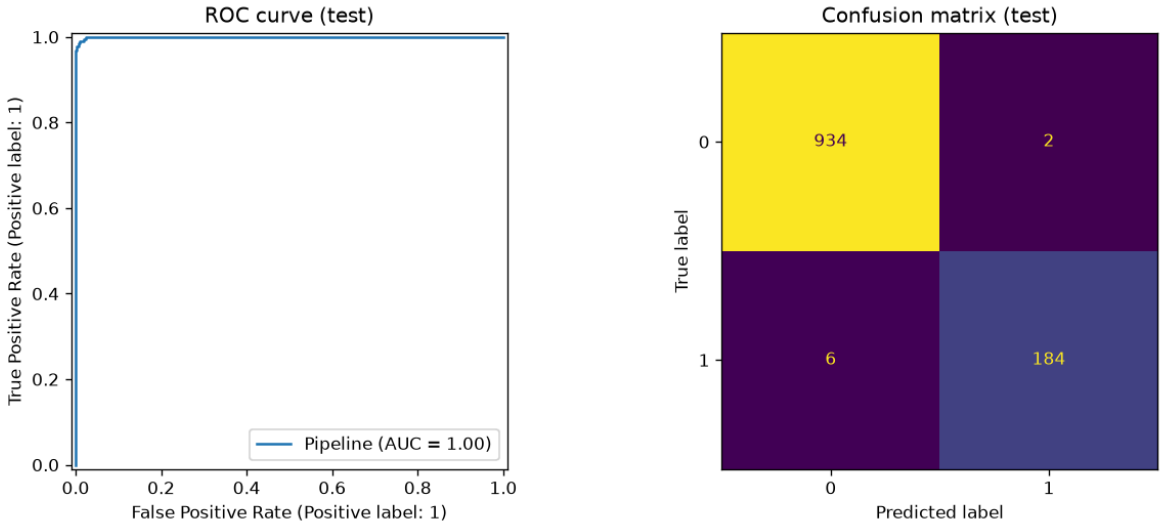
Run	CV ROC-AUC	Test ROC-AUC	Test PR-AUC	Test F1	Test recall
logreg_C1.0	0.902	0.894	0.696	0.627	0.837
random_forest_n_estimators300_max_depth8	0.947	0.963	0.863	0.737	0.863
random_forest_n_estimators500_max_depthNone	0.975	0.999	0.995	0.959	0.979
xgboost_n_estimators200_max_depth3_learning_rate0.1	0.945	0.961	0.868	0.775	0.726
xgboost_n_estimators400_max_depth5_learning_rate0.05	0.970	0.997	0.984	0.921	0.884
xgboost_n_estimators600_max_depth6_learning_rate0.05 (champion)	0.974	0.999	0.997	0.979	0.968

We picked the champion using cross-validation AUC and not the test set, so the test set stays a fair final check. We also restricted the choice to the XGBoost runs, because that is the model we explain with SHAP's TreeExplainer in Part 3.

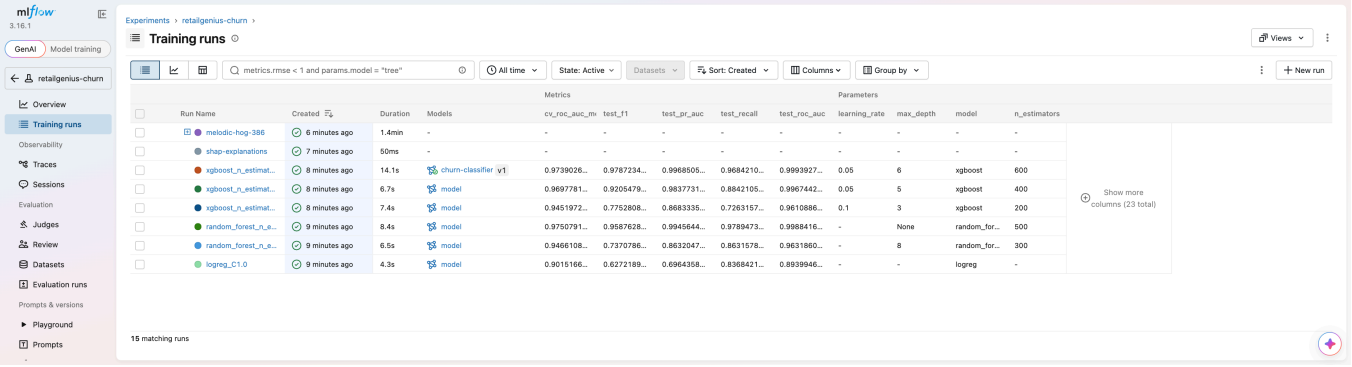
The winner is `xgboost_n_estimators600_max_depth6_learning_rate0.05`: XGBoost with 600 trees, max depth 6 and learning rate 0.05. It is registered as `churn-classifier` version 2 with the `@champion` alias. Version 1 is the same configuration from the standalone `make sweep` run; version 2 comes from the `mlflow run` project run and replaced it. Its CV AUC is 0.974, the best of the XGBoost runs. On the test set it reaches a ROC-AUC of 0.999, a PR-AUC of 0.997, an F1 of 0.979 and a recall of 0.968. In the confusion matrix, it catches 184 of the 190 churners and raises only 2 false alarms among 936 loyal customers.

The unbounded Random Forest has a slightly higher CV AUC (0.975). But it is about 20 times larger on disk and it isn't the model family we explain in Part 3, so we kept XGBoost.

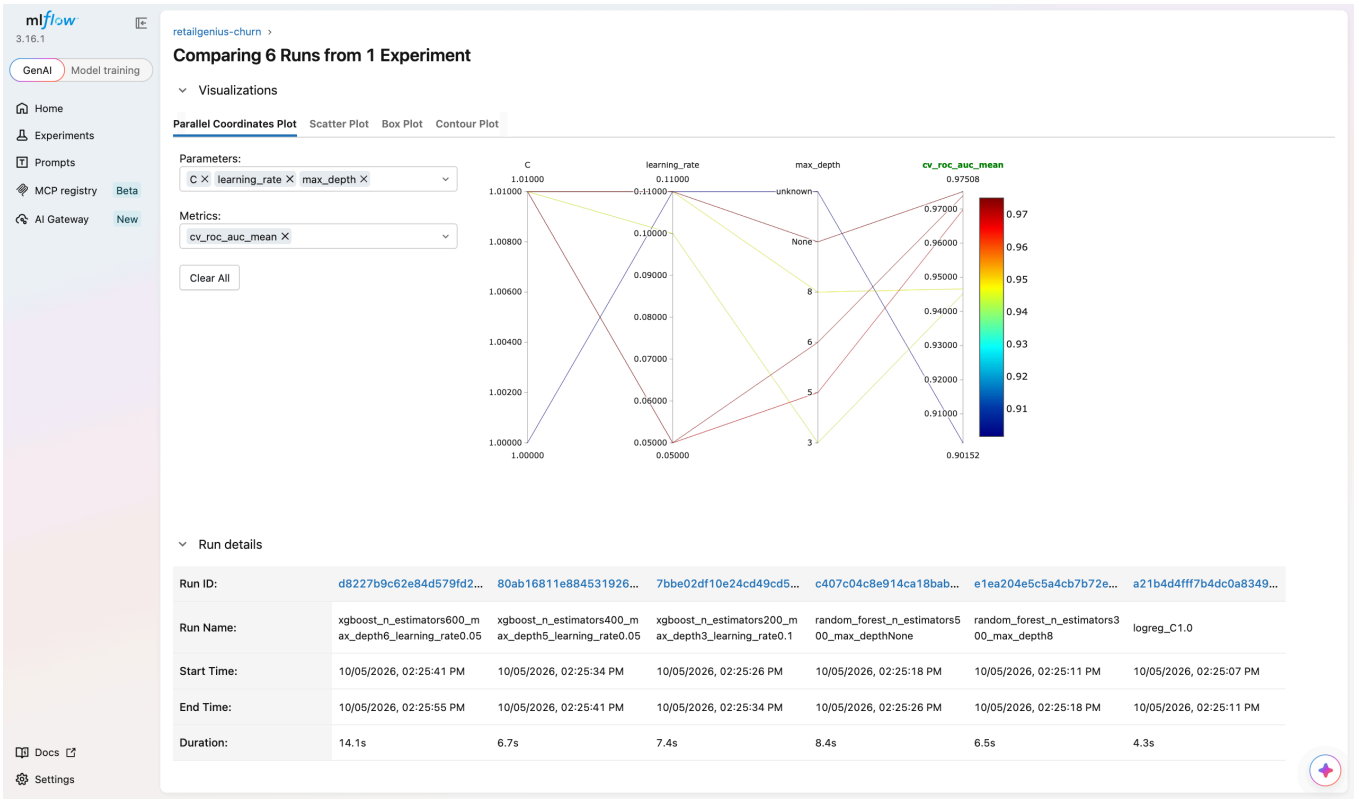
These scores are much higher than what we would expect in real life, and we think we know why. In this Kaggle dataset, 557 rows are exact duplicates except for `CustomerID`, and 194 of the 1,126 test customers have a profile that also appears in the training set. A random split is therefore too optimistic. In a real project we would remove duplicate customers before splitting, and use an out-of-time split: train on past months, test on the latest month.



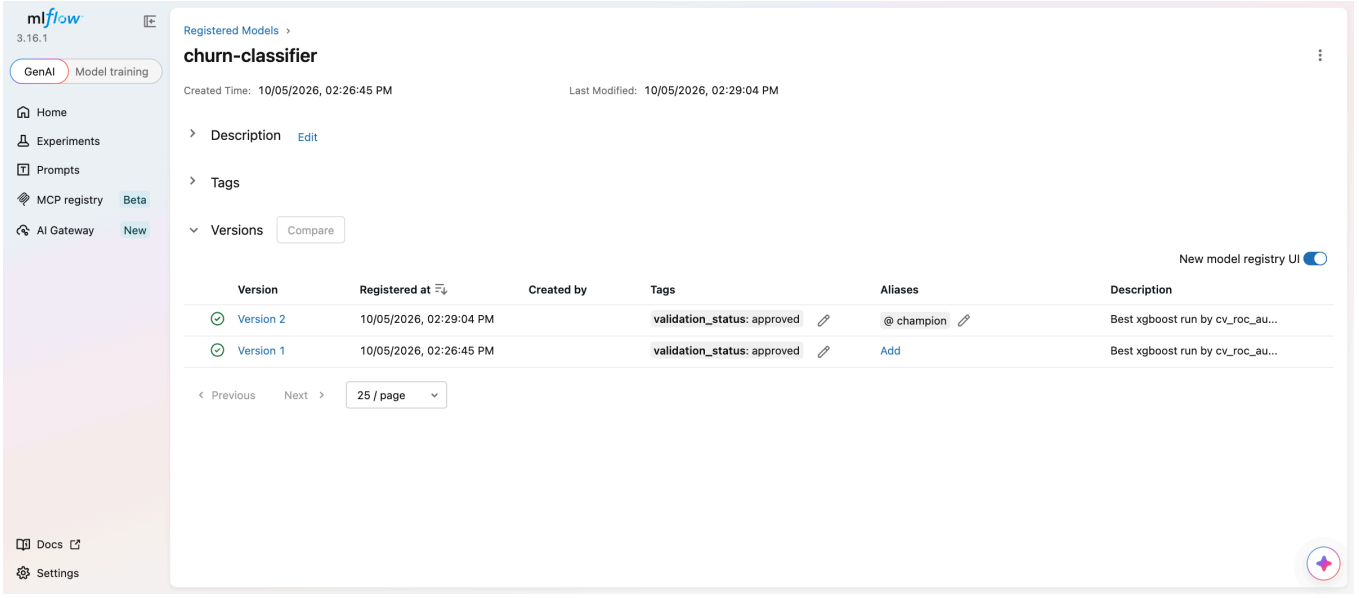
Evaluation of the champion model on the test set



MLflow UI – runs of the `retailgenius-churn` experiment with their parameters and metrics



MLflow UI – comparison of the six sweep runs (parallel coordinates of hyper-parameters against CV AUC)



MLflow Model Registry – `churn-classifier`, version 2 has the `@champion` alias and `validation_status: approved`

5. MLflow Projects, Registry and Serving

These are the main commands we used:

```
# the whole pipeline as an MLflow Project
mlflow run . -e main --env-manager local --experiment-name retailgenius-churn
# a single training run with parameters
mlflow run . -e train -P model=xgboost -P max_depth=4 --env-manager local \
           --experiment-name retailgenius-churn
# registry: best run -> churn-classifier, alias @champion
python -m churn.register
# local inference server
```

```
mlflow models serve -m "models:/churn-classifier@champion" -p 5001 --env-manager local
curl -X POST http://127.0.0.1:5001/invocations -H "Content-Type: application/json" \
     -d @data/sample_request.json
```

Here is the server's answer for three test customers. Each pair is `[P(stay), P(churn)]`:

```
{"predictions": [[0.9982208013534546, 0.0017792237922549248], [0.9997668862342834, 0.00023312029
952649027], [0.9997625350952148, 0.00023747199156787246]]}
HTTP 200
```

All three are real loyal customers (label 0), and the model scores them that way. The second one has a missing `DaySinceLastOrder`, which the pipeline fills in at serving time without any problem.

```
# make serve  -> mlflow models serve -m "models:/churn-classifier@champion" -p 5001
$ curl -s -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:5001/invocations -H "Content-Type: application/json" -d
@data/sample_request.json
{"predictions": [[0.9982208013534546, 0.0017792237922549248], [0.9997668862342834, 0.00023312029952649027],
[0.9997625350952148, 0.00023747199156787246]]}
HTTP 200
```

The local MLflow REST server answering the curl request (HTTP 200)

When we launch the pipeline with `mlflow run`, the training runs show up as nested runs under the project run, so one full execution stays grouped together in the UI.

Part 3 – Explainable AI with SHAP

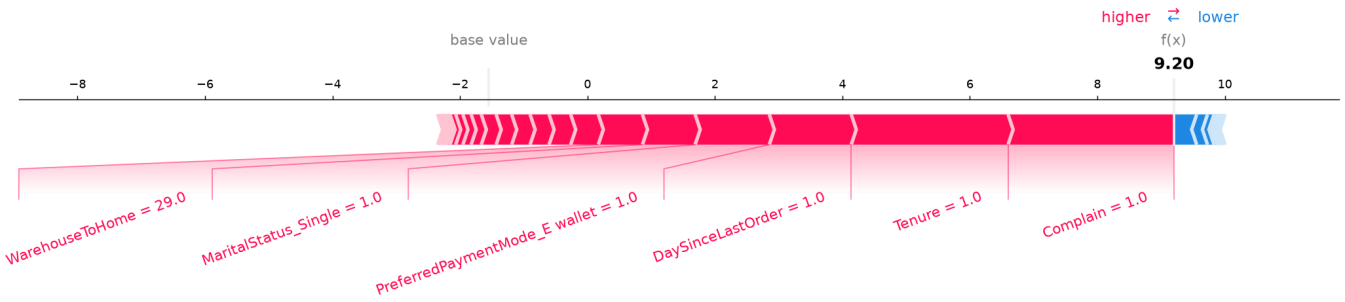
A churn score on its own isn't very useful to the people who have to act on it. A support agent needs to know *why* a customer is at risk before calling them. In this part we use SHAP to open up our champion model.

1. Setup

- Library: `shap==0.52.0`, added to `pyproject.toml` (Poetry) and to `requirements.txt`.
- Model: the champion from the Model Registry, `models:/churn-classifier@champion` (XGBoost, 600 trees, depth 6). We load it straight from MLflow, so we are explaining exactly the model that is served.
- Explainer: `shap.TreeExplainer`, which computes exact Shapley values for tree models. The values are in log-odds of churn. A positive value pushes a customer towards churning, a negative one towards staying.
- Data: the 1,126 customers of the test set, after the pipeline's own feature engineering and preprocessing. The features in the plots are therefore the ones the model really sees, for example `MaritalStatus_Single` after one-hot encoding, or our engineered `CashbackPerOrder`.
- Code: `src/churn/explain.py`, run with `make explain` (it is also the `explain` entry point in the `MLproject`). Every figure is saved in `reports/figures/shap/` and also logged to MLflow under a `shap-explanations` run, next to the training runs.
- Base value: on average the model outputs E[f(X)] = −1.544 log-odds, which is about a 17.6 % churn probability. That is close to the real churn rate of 16.8 %.

The customer we explain. For the single-customer plots, we took the real churner with the highest predicted risk in the test set (test row 404, `CustomerID` 54288). He is single, lives in a tier-3 city and has been a customer for only one month. He has already filed a complaint and placed his last order one day ago. He pays with an e-wallet, lives 29 km from the warehouse and mostly buys mobile phones. The model gives him a churn probability of almost 1.00 (f(x) = 9.20 log-odds), and he did churn.

2. Explaining one customer (force plot)

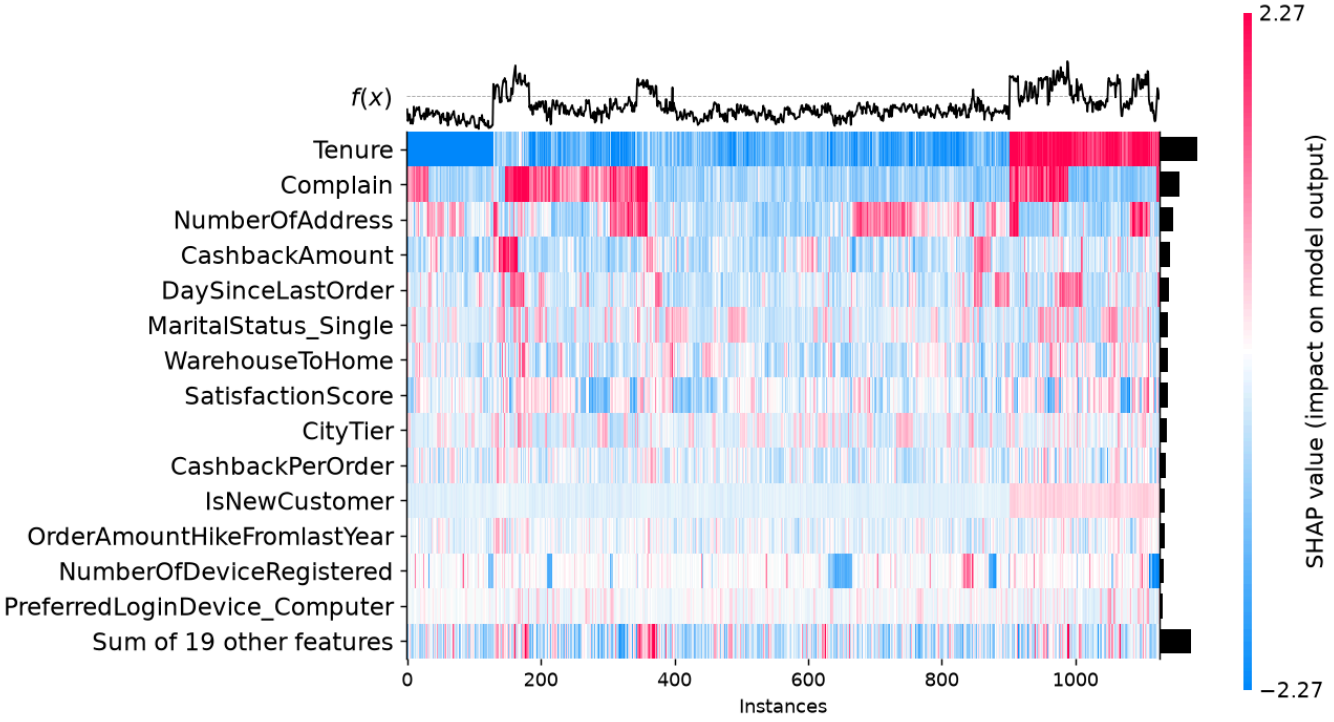


Force plot for the explained customer

The force plot starts from the base value (−1.54) and shows which features push the prediction up (red) or down (blue), until it reaches f(x) = 9.20. Almost everything is red. The two biggest pushes come from Complain = 1 and Tenure = 1. Next come DaySinceLastOrder = 1, paying with an e-wallet, being single and the 29 km delivery distance. Only a few small blue arrows, like his 20 % increase in order amount, pull a little towards staying.

In short, he is at risk because he is a new customer who is already unhappy: he complained during his first month. An interactive version of this plot is in `01_force_plot_single.html`.

3. Explaining all customers at once



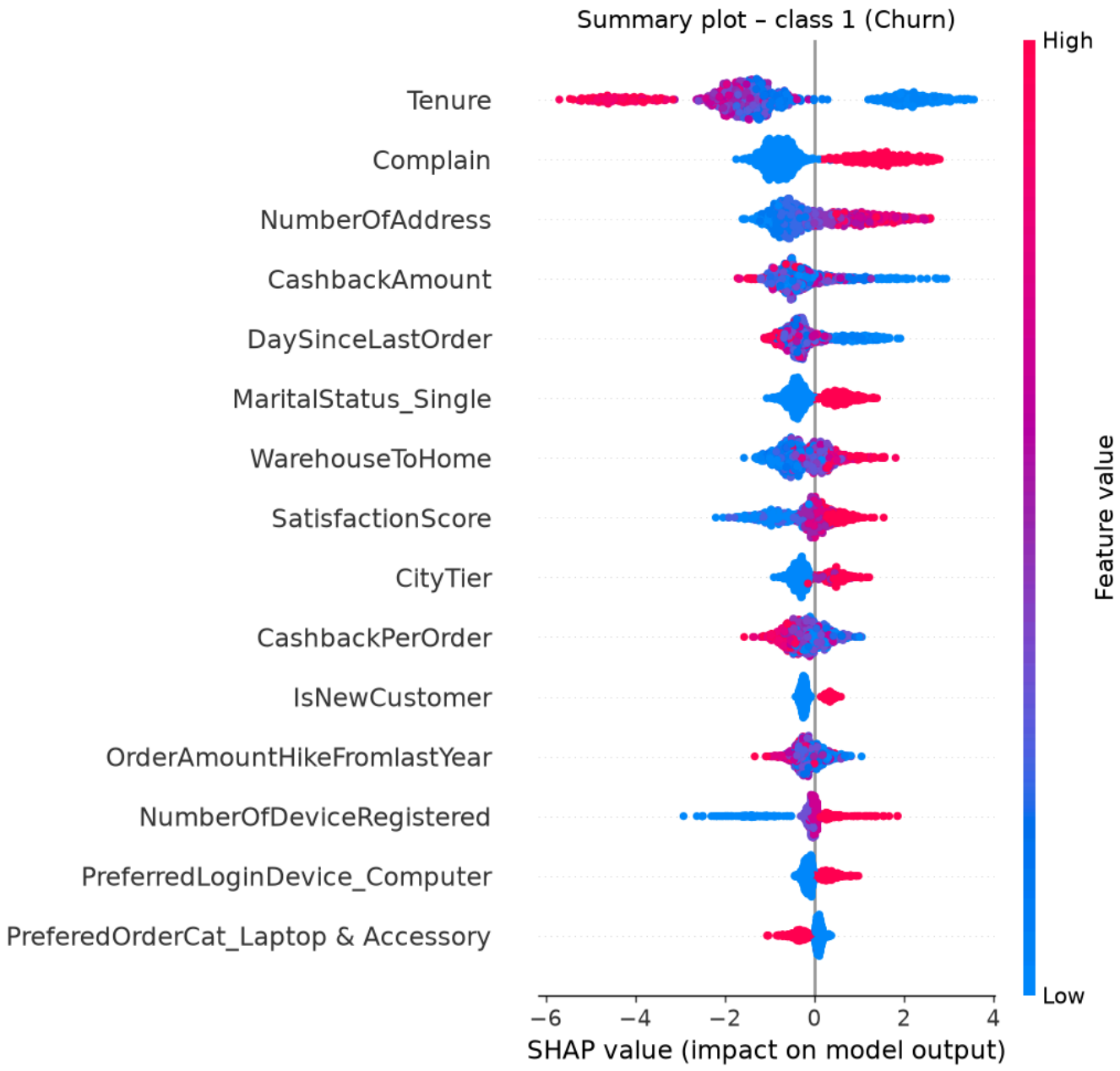
SHAP heatmap of all test customers

The heatmap has one column per test customer (1,126 of them) and one row per feature, coloured by SHAP value. Customers are grouped by how similar their explanations are, and the model output f(x) is drawn on top.

Tenure is clearly the dominant row. The group of customers on the right (around columns 900 to 1,120) has strongly red Tenure, meaning very new customers (`IsNewCustomer` is red there too), and the highest risk. A second high-risk group (around columns 140 to 360) is driven mostly by Complain. So the model sees two quite different churn profiles: newcomers and complainers.

There is also an interactive stacked force plot of all 1,126 customers in `02_force_plot_all.html`. It can be sorted by similarity, by output value or by any feature.

4. Summary plot for each class



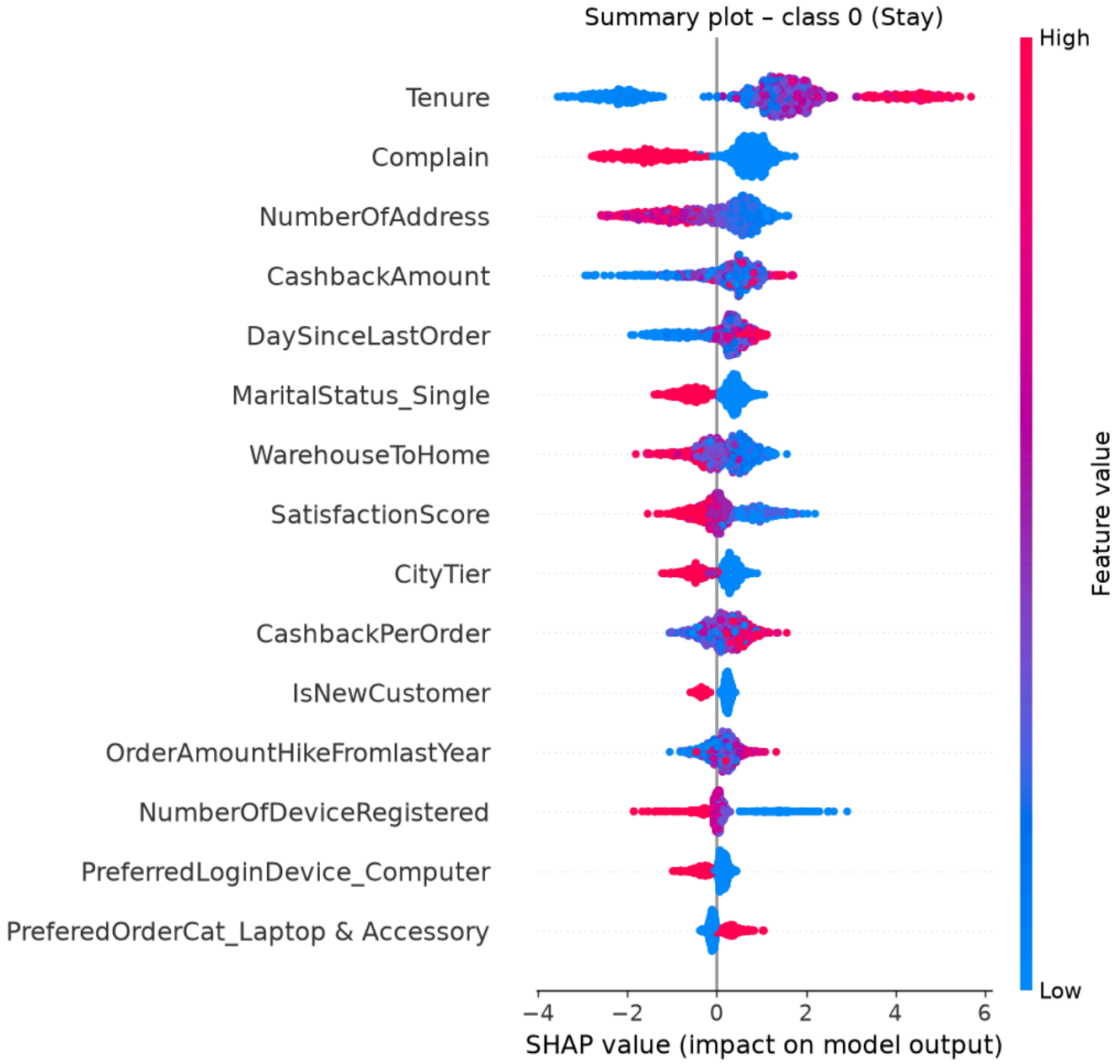
Summary plot – class 1 (Churn)

For class 1 (churn), each dot is a customer. Its horizontal position is the SHAP value and its colour is the feature value (red for high, blue for low). Low tenure (blue) gives large positive values, up to +3.5, while long tenure gives strongly negative ones, down to −6. Several other things push towards churn:

- having complained;
- having many addresses;
- receiving little cashback;
- having ordered recently;
- being single;
- living far from the warehouse;
- living in a higher city tier;
- having many registered devices.

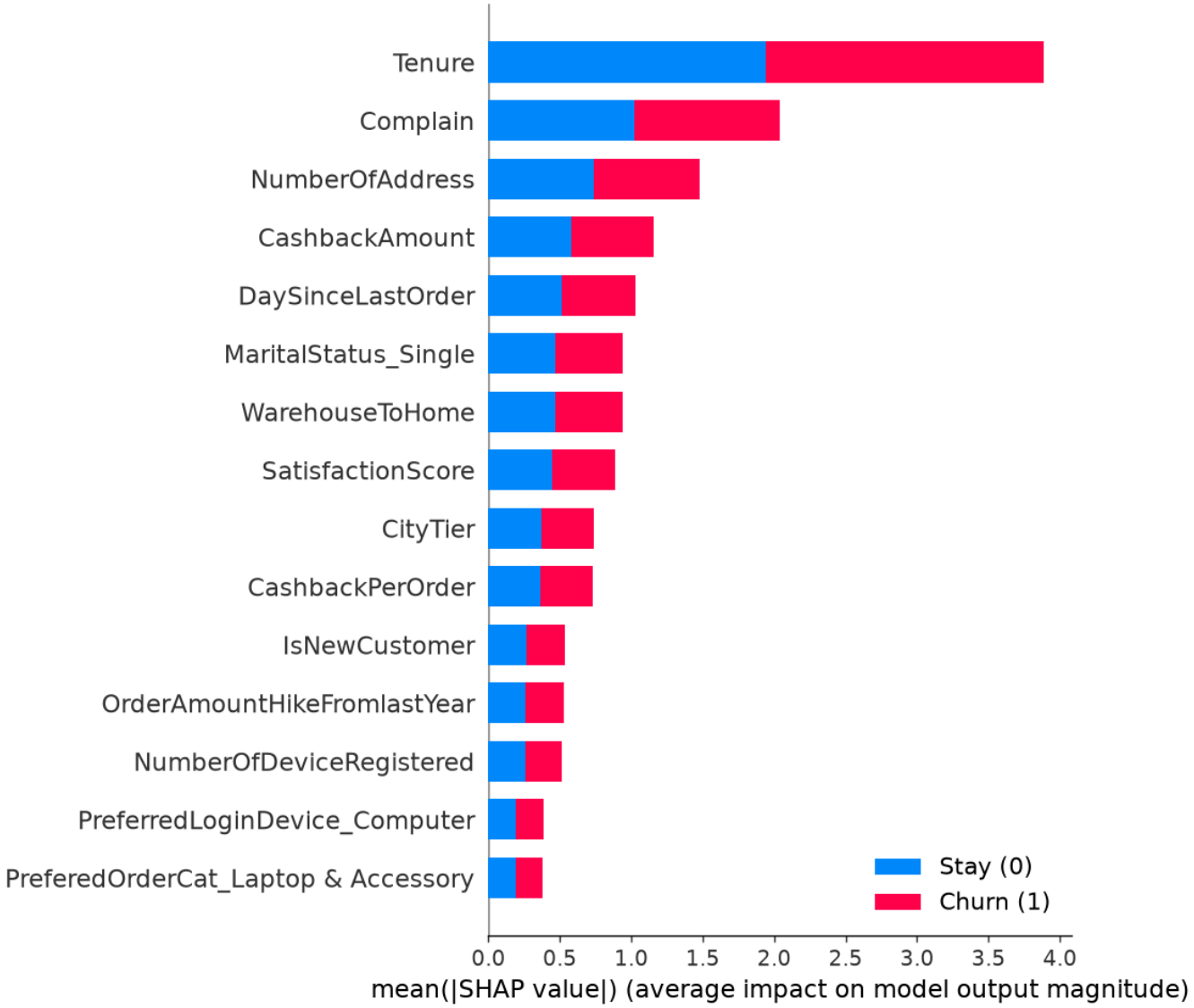
Two effects surprised us: churners tend to have ordered *recently* (low `DaySinceLastOrder`), and a *higher* satisfaction score slightly increases the risk. Both are consistent across the dataset, but we

see them as model behaviour that should be checked with the business, not as facts about customers.



Summary plot – class 0 (Stay)

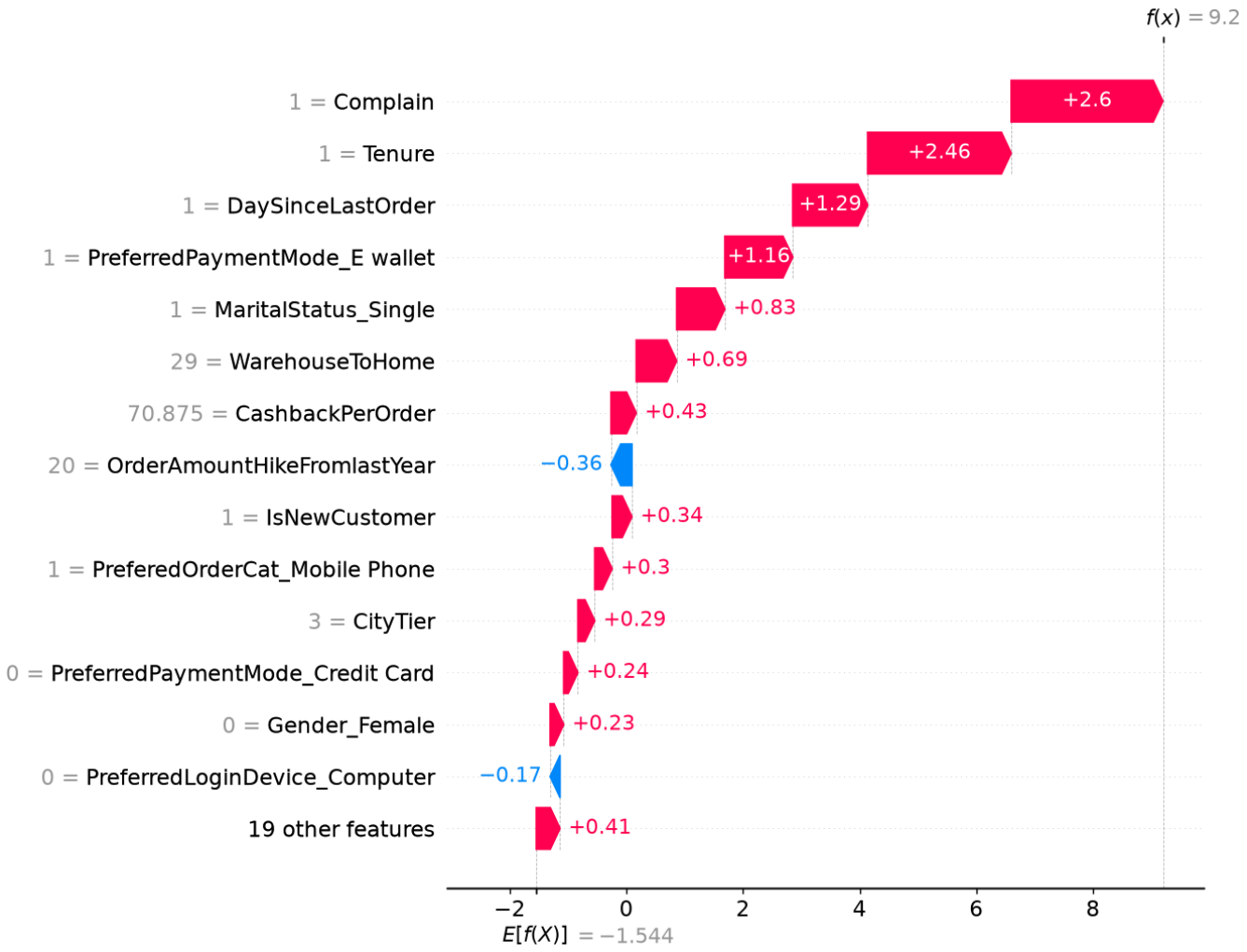
For class 0 (stay), the plot is an exact mirror of class 1. In a binary model the log-odds of staying are just the negative of the log-odds of churning, so every SHAP value flips sign. Long tenure and having no complaint now become the strongest reasons to stay. The order of the features doesn't change.



Mean |SHAP| per class

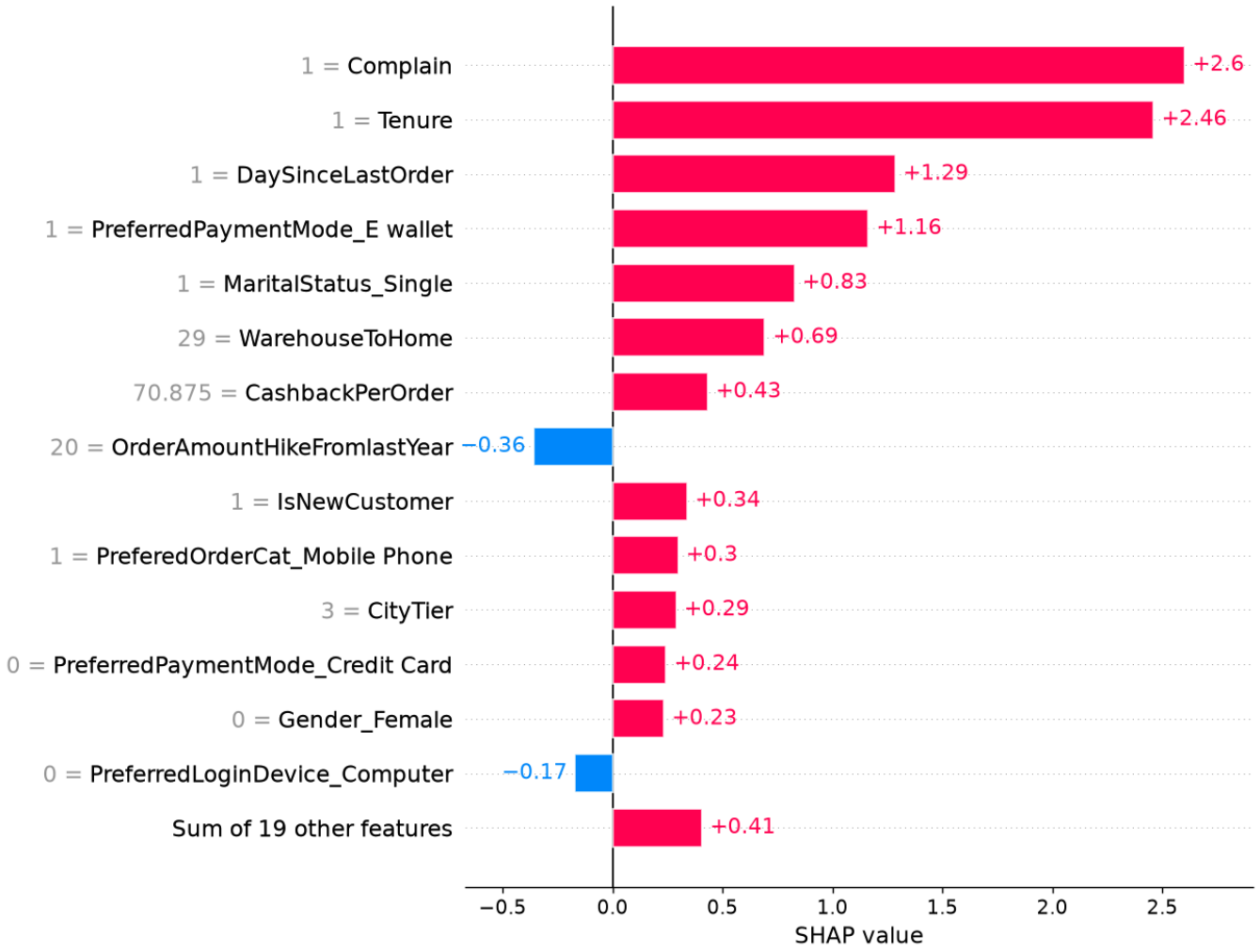
The bar chart per class confirms this: each feature has the same mean |SHAP| for both classes, so the blue and red parts are equal. The overall ranking is Tenure, then Complain, NumberOfAddress, CashbackAmount and DaySinceLastOrder.

5. Going further for one customer: waterfall plot



Waterfall plot for the explained customer

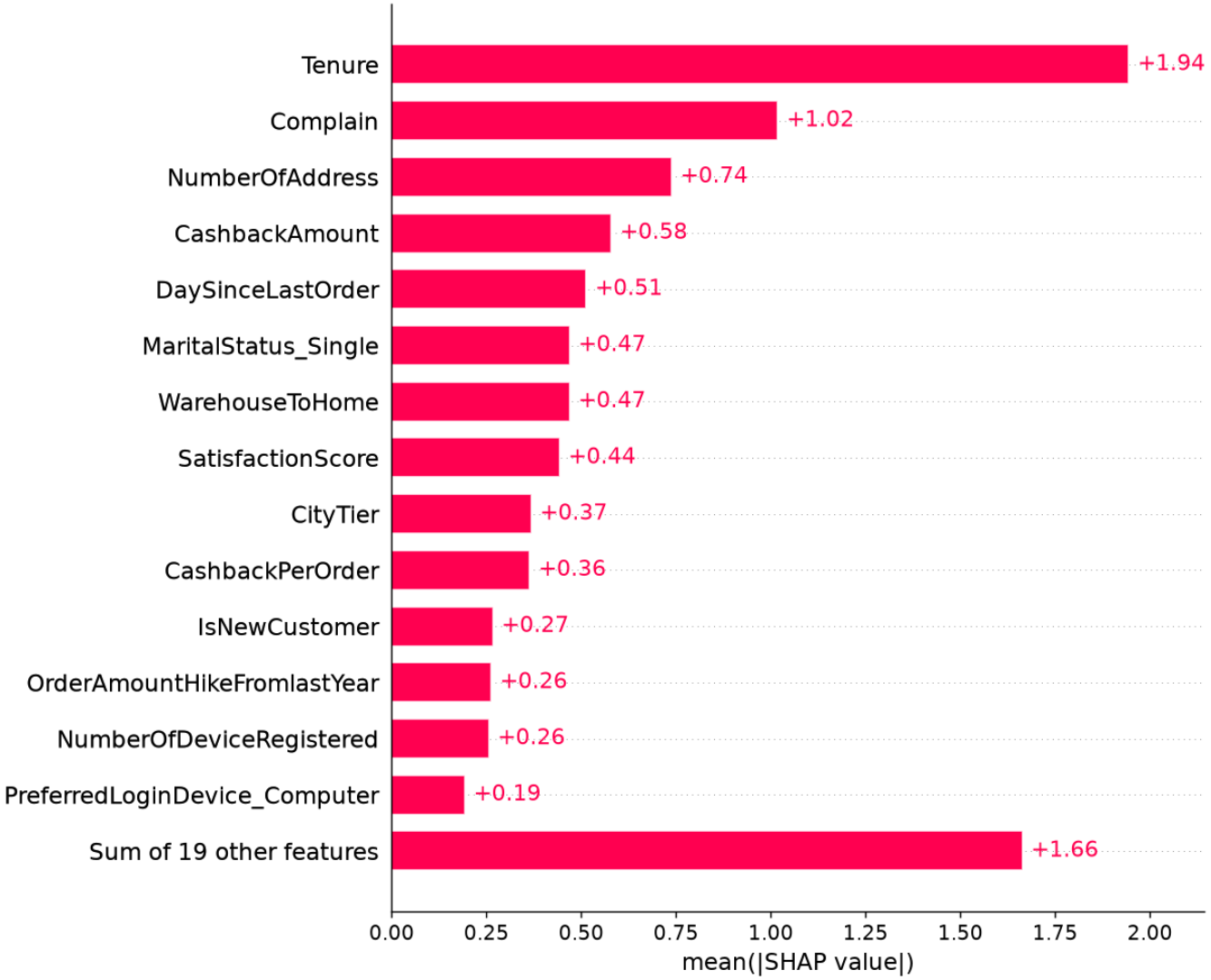
The waterfall plot tells the same story as the force plot, but with exact numbers. It goes from E[f(X)] = −1.544 at the bottom to f(x) = 9.2 at the top. Complain (+2.60) and Tenure (+2.46) alone add more than 5 log-odds. After that come DaySinceLastOrder (+1.29), paying with an e-wallet (+1.16), being single (+0.83), the 29 km distance (+0.69) and a high cashback per order (+0.43). Only the order amount increase (−0.36) and not using a computer to log in (−0.17) lower his risk. The 19 remaining features add +0.41 together.



Bar plot for the explained customer

This local bar plot shows the same values sorted by size. It is the easiest format to turn into the “top 3 reasons” a support agent would see: complained, new customer, ordered very recently.

6. Mean SHAP plot (global importance)



Mean |SHAP| over the test set

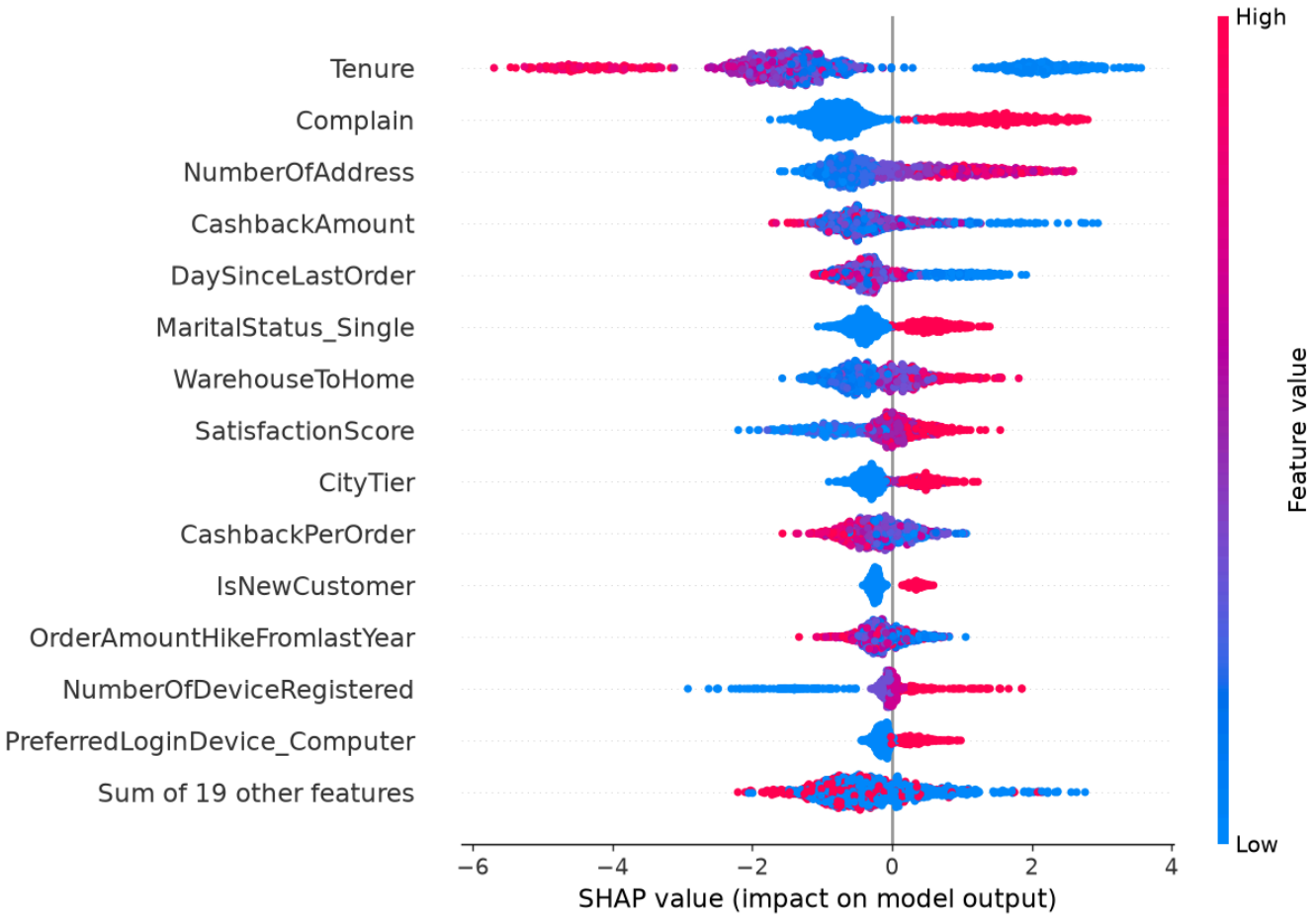
The mean absolute SHAP value measures how much a feature moves the prediction on average, whatever the direction. Tenure (1.94) matters about twice as much as Complain (1.02). Next come NumberOfAddress (0.74) and CashbackAmount (0.58). Unlike the built-in “gain” importance of XGBoost, this measure is consistent and is expressed in log-odds. The 19 smallest features still add up to 1.66, so a lot of small signals matter together.

Top 10 features (from `reports/figures/shap/mean_abs_shap.csv`):

Rank	Feature	Mean |SHAP| (log-odds)
1	Tenure	1.9447
2	Complain	1.0189
3	NumberOfAddress	0.7385
4	CashbackAmount	0.5793
5	DaySinceLastOrder	0.5136
6	MaritalStatus_Single	0.4694
7	WarehouseToHome	0.4691
8	SatisfactionScore	0.4429

Rank	Feature	Mean |SHAP| (log-odds)
9	CityTier	0.3680
10	CashbackPerOrder	0.3644

7. Beeswarm plot

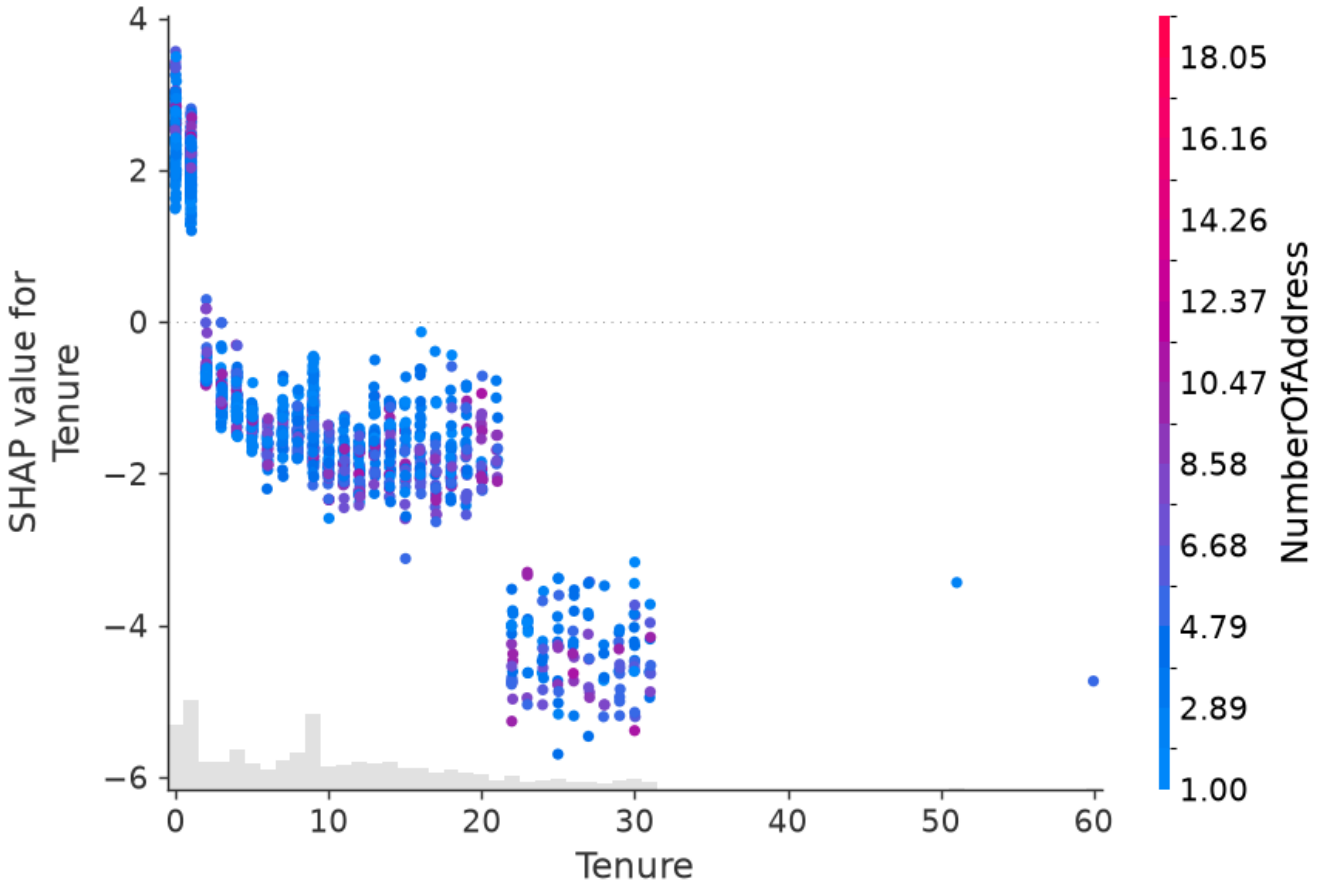


Beeswarm plot

The beeswarm plot puts importance (the vertical order) and direction (the horizontal position and colour) together for every customer:

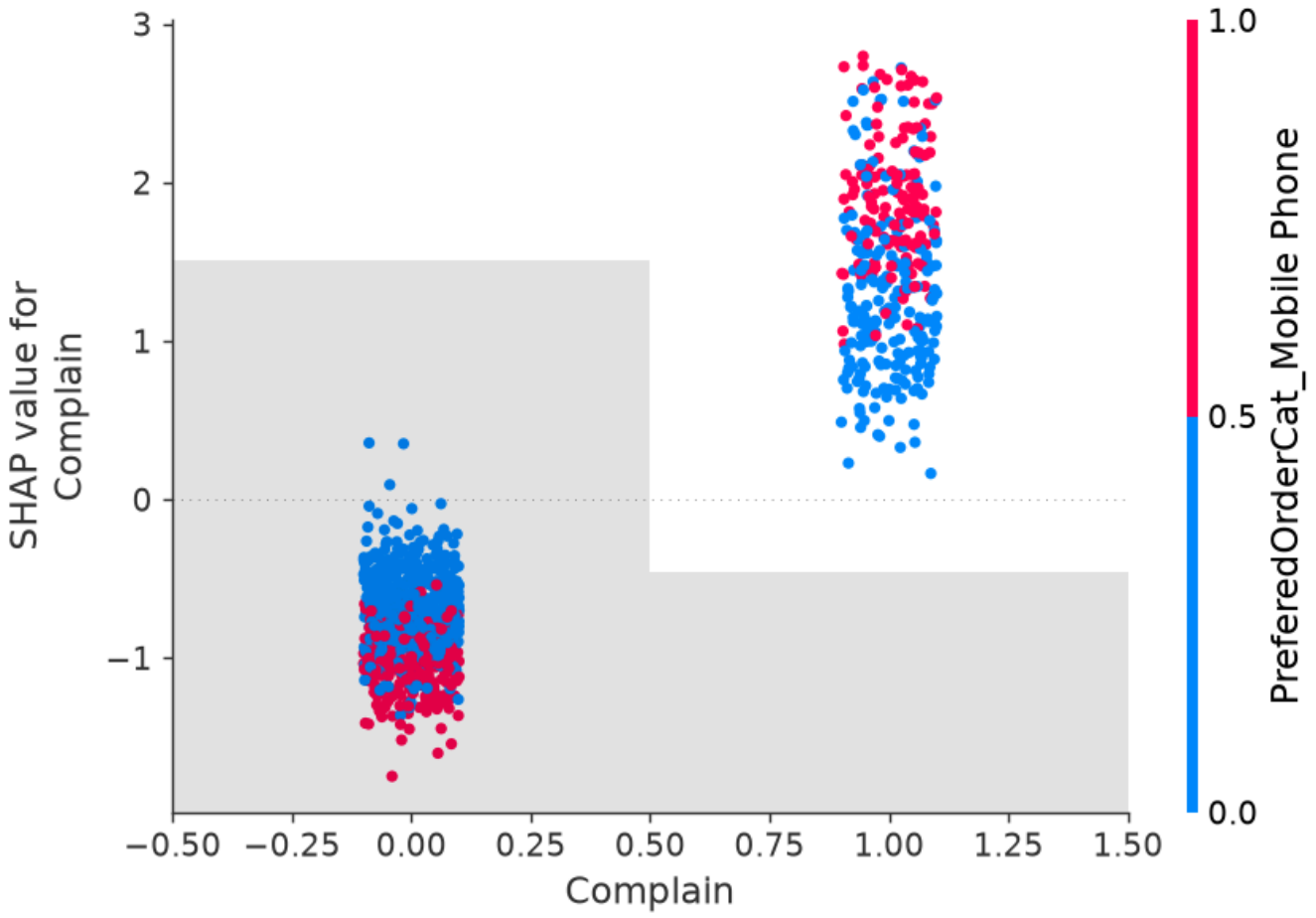
- Tenure has the widest spread of all, from about −6 to +3.5.
- Complain splits customers into two clean groups.
- NumberOfDeviceRegistered has a long blue tail on the left, which means customers with only a few devices are clearly less likely to churn.
- Our engineered `IsNewCustomer` feature adds a small but clear positive effect on top of Tenure.

8. Dependence plots (top 4 features)



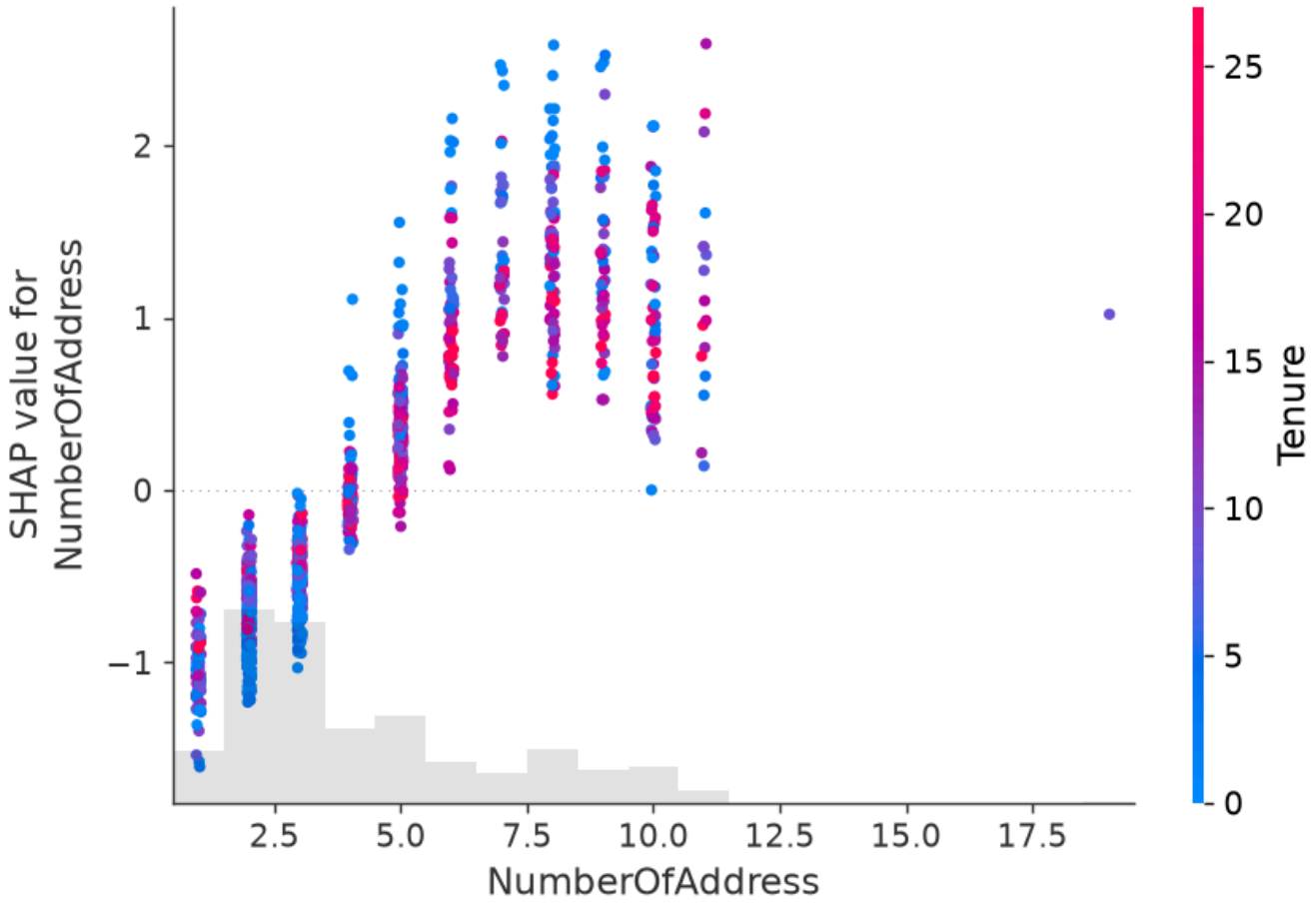
Dependence – Tenure

Tenure. The effect looks like a staircase. Customers with 0 or 1 month of tenure get +1.2 to +3.6. From about two months the effect drops below zero (−1 to −2), and customers with 22 to 31 months get −3.5 to −5.5. The first month is clearly the critical moment for retention. The colour (NumberOfAddress) doesn't show a strong interaction.



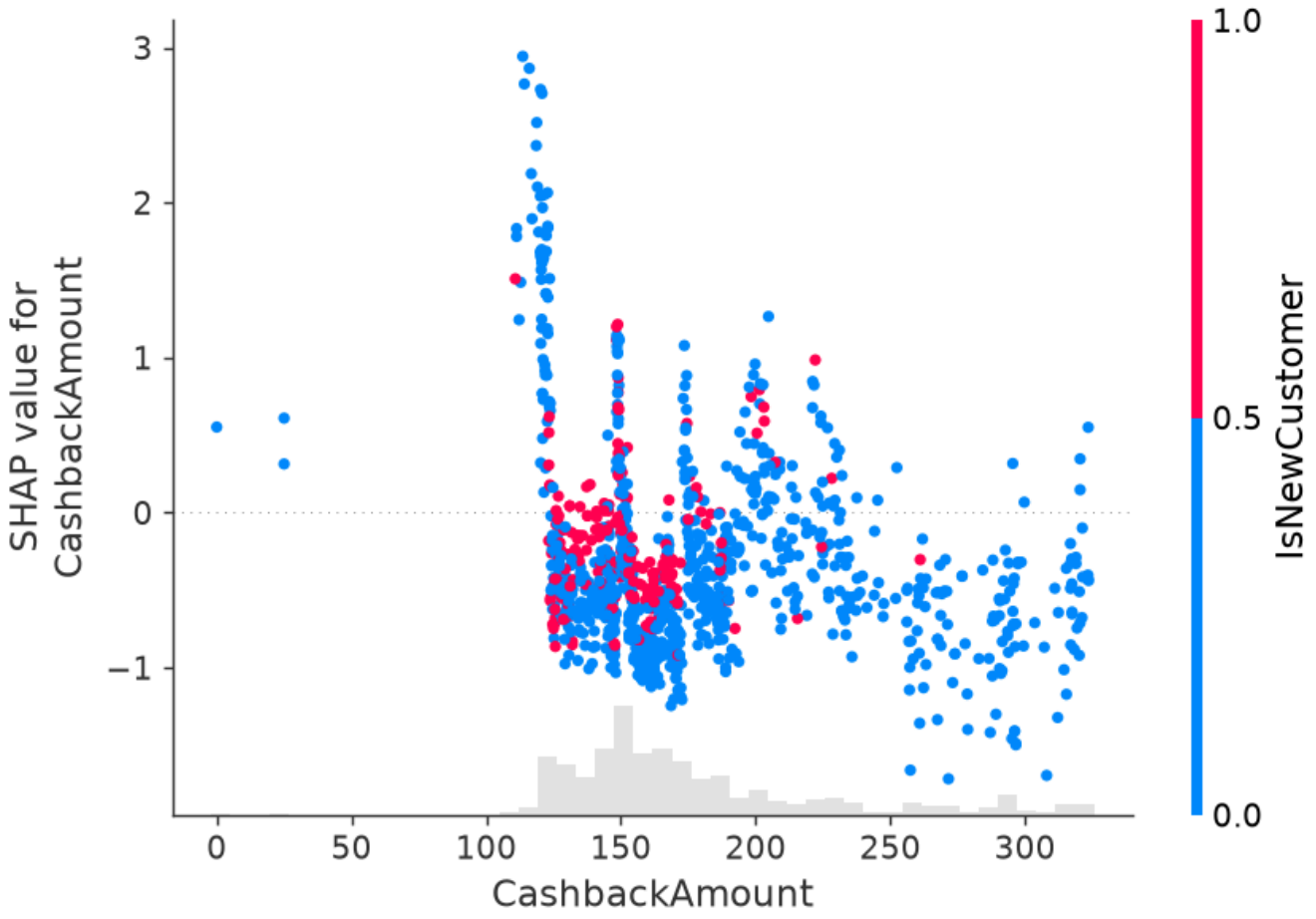
Dependence – Complain

Complain. This is a simple jump: about −0.5 to −1.7 without a complaint, and +0.2 to +2.8 with one. Among the customers who complained, those whose favourite category is Mobile Phone (in red) tend to get the highest values. Complaints about phone orders seem to be the most dangerous.



Dependence – NumberOfAddress

NumberOfAddress. One to three addresses lower the risk (−0.2 to −1.6). The effect crosses zero around four or five addresses and reaches +1 to +2.5 for seven to ten. Many delivery addresses could point to a shared or less settled account, which would be worth checking with the business.



Dependence – CashbackAmount

CashbackAmount. Customers who received less than about 125 in cashback get strongly positive values (up to +3). Between 125 and 200 the effect is mostly negative, and above 250 it is always negative. The vertical spikes at some amounts come from the trees splitting on the many identical rows in this dataset. Overall, more cashback goes with more loyalty, which makes cashback a natural lever for retention.

9. What this means for the business

SHAP turns a churn score into a reason we can act on, customer by customer. The daily batch job could store each customer's churn probability together with their top three SHAP reasons, written in plain language, in the CRM. Marketing and Customer Support then pick the action that fits the reason:

Main SHAP reason	Suggested action	Team
`Complain = 1`	A call-back from a senior agent, make sure the ticket is solved, a goodwill gesture	Customer Support
Low `Tenure` / `IsNewCustomer`	An onboarding journey: a welcome coupon for the second order, an app tutorial, a check-in after the first month	Marketing (CRM)
Low `CashbackAmount`	A targeted cashback offer instead of a generic discount	Marketing
Long `WarehouseToHome`	A free or faster delivery offer, and feedback to logistics	Marketing / Operations
High `NumberOfAddress`	Check for account sharing or delivery problems	Customer Support

Looking at the whole customer base, the plots show where the real problems are. Churn is concentrated in the first month and among customers who complained. Onboarding and complaint handling are therefore the two processes worth improving first, and SHAP gives us a way to see whether their weight goes down after the change.

Explanations also build trust. Agents are much more willing to act on a score they understand. They also make it possible to answer a customer or a regulator who asks about an automated decision (GDPR, Article 22).

We should be careful about one thing, though: SHAP explains the model, not the real world. A positive SHAP value for `Complain` means the model relies on complaints to predict churn. It doesn't prove that preventing complaints would stop people from leaving. Some effects, like a recent order or a higher satisfaction score increasing the risk, are probably side effects of how this dataset was built. Correlated features also share the credit between them, such as Tenure and IsNewCustomer, or CashbackAmount and CashbackPerOrder. Any causal claim, and the real effect of each retention action, would have to be tested with A/B tests (a treated group compared with a control group), as we planned in Part 1.

References

Explainable AI

1. Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems 30 (NeurIPS 2017), pp. 4765–4774. https://arxiv.org/abs/1705.07874
2. SHAP documentation. https://shap.readthedocs.io/

Models and libraries

3. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794. https://doi.org/10.1145/2939672.2939785
4. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research 12, pp. 2825–2830. Documentation: https://scikit-learn.org/stable/

MLOps and tooling

5. MLflow documentation: Tracking https://mlflow.org/docs/latest/ml/tracking/, Projects https://mlflow.org/docs/latest/ml/projects/, Model Registry https://mlflow.org/docs/latest/ml/model-registry/, Model serving https://mlflow.org/docs/latest/ml/deployment/
6. DrivenData. *Cookiecutter Data Science*. https://cookiecutter-data-science.drivendata.org/
7. Sphinx documentation generator. https://www.sphinx-doc.org/
8. Poetry: Python packaging and dependency management. https://python-poetry.org/docs/
9. van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style Guide for Python Code*. https://peps.python.org/pep-0008/; black https://black.readthedocs.io/; flake8 https://flake8.pycqa.org/

Data

10. Verma, A. *E-Commerce Customer Churn Analysis and Prediction* (dataset). Kaggle. https://www.kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-prediction

Project methodology

11. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
12. Trello (Atlassian), Kanban board tool. https://trello.com/